



Tecnológico Nacional de México Campus Culiacán

Ingeniería en Sistemas Computacionales

Materia:

Ingeniería de Software

Tares Bonus Modulo 2

Profesora:

Zuriel Dathan mora Félix

Alumnos:

Campos López Luis Fernando

Vega Hernández Gustavo Ángel

1. Problema de Programación de Trabajos (Job Shop Scheduling Problem - JSSP)

El JSSP es un modelo matemático y computacional que simula el entorno de una fábrica o taller. Imagina una planta de manufactura donde llegan distintos pedidos (trabajos) y cada pedido requiere pasar por varias estaciones especializadas (máquinas). El problema radica en que no puedes procesar todo al mismo tiempo; debes organizar un "horario" o "itinerario" perfecto para que las máquinas no se queden inactivas esperando y los trabajos se terminen lo más rápido posible.

Es considerado un problema **NP-difícil**, lo que significa que a medida que aumentas el número de máquinas y trabajos, el número de combinaciones posibles crece de forma exponencial, haciendo imposible encontrar la solución perfecta por fuerza bruta en un tiempo razonable.

Reglas y Limitaciones (Restricciones)

El JSSP es muy estricto en sus reglas operativas, divididas en tres categorías principales:

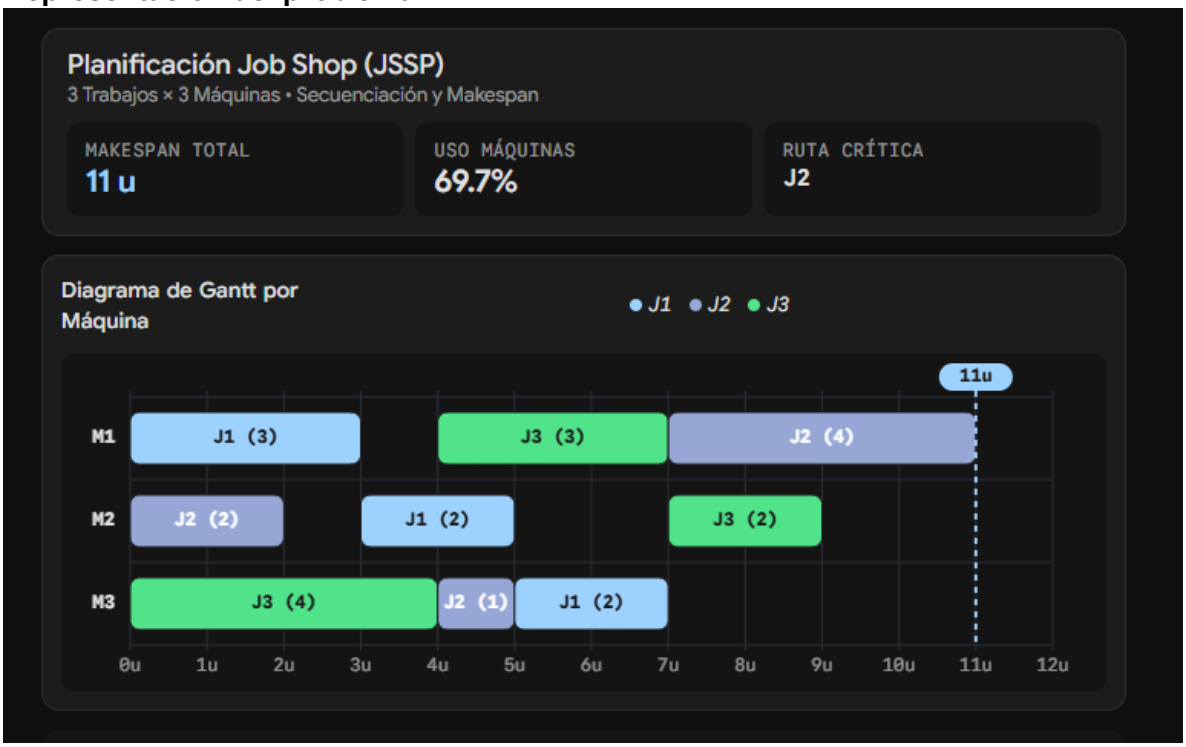
1. **Restricción de Precedencia (Rutas):** Cada trabajo J_i tiene una secuencia tecnológica inalterable. Por ejemplo, el Trabajo 1 debe pasar por la Máquina 3, luego por la Máquina 1 y finalmente por la Máquina 2. No puedes alterar ese orden. La operación $O_{i,2}$ no puede iniciar hasta que $O_{i,1}$ haya finalizado por completo.
2. **Restricción de Capacidad de Máquina:** Una máquina M_k solo puede procesar una operación a la vez. No existe el procesamiento en paralelo en la misma máquina. Si dos trabajos necesitan la Máquina 1 al mismo tiempo, uno debe esperar en una cola.
3. **Restricción de No-Interrupción (No-Preemption):** Una vez que una operación comienza a ejecutarse en una máquina, debe continuar ininterrumpidamente hasta terminar su tiempo de procesamiento (p_{ij}). No puedes pausar un trabajo a la mitad para meter un trabajo con mayor prioridad.

Objetivo Principal

El objetivo matemático universal del JSSP es **minimizar el *Makespan* (C_{max})**.

El makespan se define como el tiempo total transcurrido desde que se inicia la primera operación del primer trabajo en el instante $t = 0$, hasta que finaliza la *última* operación del *último* trabajo. Minimizar el makespan significa que la fábrica completó todos sus pedidos en el menor tiempo posible, optimizando el uso de recursos y reduciendo los tiempos muertos.

Representación del problema



En la representación del problema mediante un Diagrama de Gantt, se simula el escenario de un taller clásico que debe procesar 3 trabajos (J1, J2, J3) utilizando 3 máquinas disponibles (M1, M2, M3). El diagrama ilustra gráficamente cómo se distribuyen los bloques de tiempo respetando la restricción de que cada máquina solo puede ejecutar una operación a la vez. Al organizar la secuencia de forma óptima, se puede observar que el "Makespan" total es de 11 unidades de tiempo, marcando con una línea punteada el momento exacto en que la fábrica termina la última operación del Trabajo 2 en la Máquina 1.

2. Problema de las N Reinas

¿Qué es?

Es una generalización del clásico acertijo de ajedrez de "Las 8 reinas", propuesto en el siglo XIX. El problema consiste en intentar colocar N reinas en un tablero de ajedrez modificado de tamaño $N \times N$, de forma que estén en paz, es decir, que ninguna amenace a otra.

En ciencias de la computación, no es solo un juego; es el ejemplo de libro de texto para enseñar algoritmos de búsqueda exhaustiva sistemática, específicamente el **Backtracking (Vuelta atrás)**. Representa un problema puro de satisfacción de restricciones donde no buscas un valor mínimo o máximo, sino simplemente una configuración que sea válida según las reglas.

Reglas y Limitaciones (Restricciones)

Basado en los movimientos de la reina en el ajedrez (que puede moverse cualquier cantidad de casillas en línea recta horizontal, vertical o diagonal), las restricciones para que dos reinas no se ataquen son:

1. **Restricción de Fila:** No puede haber más de una reina en la fila i.
2. **Restricción de Columna:** No puede haber más de una reina en la columna j.
3. **Restricción de Diagonales Principales (\):** Dos reinas ubicadas en las posiciones (r_1, c_1) y (r_2, c_2) no pueden cumplir que $r_1 - c_1 = r_2 - c_2$.
4. **Restricción de Diagonales Secundarias (/):** Dos reinas no pueden cumplir que $r_1 + c_1 = r_2 + c_2$.

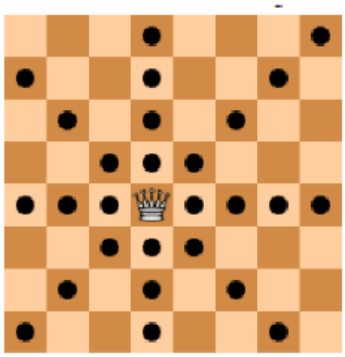
Nota técnica: Computacionalmente, en lugar de revisar todo el tablero, si usas un arreglo donde A[i] = j significa "reina en fila i, columna j", la restricción 1 se cumple automáticamente. Solo debes validar que A[i] != A[k] (columnas distintas) y que |A[i] - A[k]| != |i - k| (diagonales distintas) para cualquier fila $k < i$.

Objetivo Principal

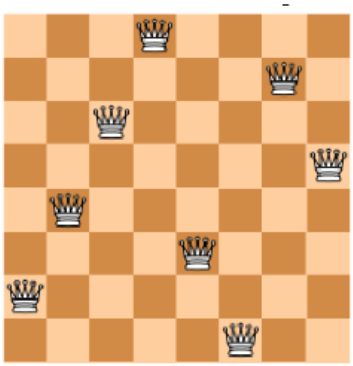
El objetivo es encontrar al menos una disposición válida de las N reinas en el tablero $N \times N$ que satisfaga todas las restricciones simultáneamente. Dependiendo del alcance de la investigación, el objetivo secundario puede ser encontrar **todas** las soluciones posibles para un N dado, lo cual es útil para analizar la complejidad temporal del algoritmo.

Representación del problema

Una reina ataca a otra reina si hay dos reinas en la misma fila, columna o diagonal. La siguiente imagen muestra los lugares en los que una reina ataca, por lo que no podría ponerse una segunda reina en cualquiera de las casillas marcadas con puntos.



El siguiente ejemplo muestra un tablero en la cual ninguna reina se ataca debido a su posición en el tablero:



La representación gráfica de este problema se ilustra utilizando un clásico tablero de ajedrez de 8x8 casillas. En la primera imagen se observan los puntos negros que marcan todas las líneas de ataque (verticales, horizontales y diagonales) de una sola reina, demostrando visualmente las celdas restringidas donde no podría colocarse otra pieza. En la segunda imagen, se muestra una representación de una solución exitosa: 8 reinas colocadas de tal forma que ninguna interfiere en las líneas de ataque de las demás, cumpliendo con la regla de no agresión en todo el tablero.

3. Árbol de Expansión Mínima (Minimum Spanning Tree - MST)

¿Qué es?

Imagina que eres un urbanista y tienes que conectar 10 ciudades a una nueva red eléctrica. Conoces el costo de tender el cable entre cada par de ciudades. No necesitas que todas las ciudades estén conectadas directamente con todas las demás; solo necesitas que, de alguna forma, la electricidad pueda llegar a todas (es decir, que haya un camino entre cualquier par de ciudades). El MST es la solución a este problema.

En teoría de grafos, dado un grafo G compuesto por vértices (nodos) y aristas (conexiones), el MST extrae el "esqueleto" más barato que mantiene todo conectado, eliminando cualquier redundancia (ciclos).

Reglas y Limitaciones (Restricciones)

Para que un subgrafo sea considerado un Árbol de Expansión Mínima válido, debe cumplir estrictamente:

1. **Conectividad total (Expansión):** El árbol debe incluir a los V vértices del grafo original. No puede quedar ningún nodo aislado.
2. **Ausencia de ciclos (Árbol):** Si el grafo original tiene V vértices, el MST debe tener exactamente $V - 1$ aristas. Si tiene más de $V - 1$ aristas, significa que hay un ciclo (un bucle cerrado), lo cual es redundante e incrementa el costo innecesariamente.
3. **Grafos conexos y no dirigidos:** El grafo original debe estar en una sola pieza (conexo) y las aristas deben permitir viaje en ambas direcciones (no dirigidas), cada una con un peso (costo) estrictamente asociado.

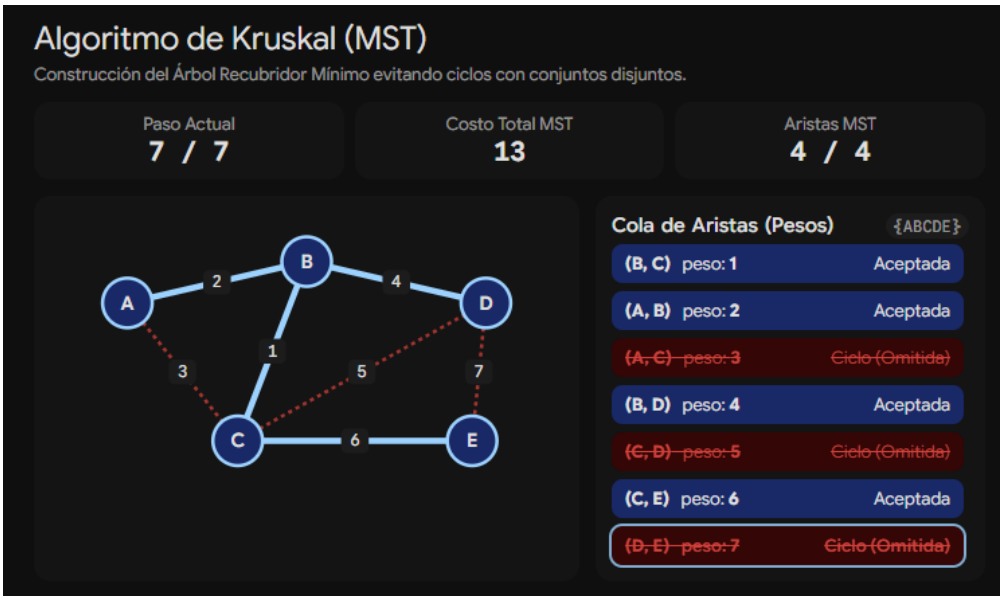
Objetivo Principal

El objetivo matemático es encontrar el subconjunto de aristas T tal que la suma de sus pesos $W(T) = \sum_{e \in T} peso(e)$ sea **la mínima posible** entre todos los árboles de expansión válidos que se puedan formar en ese grafo.

Los algoritmos para lograrlo (como Prim, que construye el árbol creciendo desde un nodo inicial, o Kruskal, que ordena todas las aristas de menor a mayor y las va uniendo siempre que no formen un ciclo) son ejemplos perfectos de algoritmos **Voraces (Greedy)**: toman la mejor decisión local en el momento y, sorprendentemente en este problema, eso garantiza la solución óptima global.

Representación del problema





Visualmente, el problema se modela a través de un grafo compuesto por nodos (letras de la A a la E) interconectados por aristas con un costo o peso específico. Como se observa en la captura del Algoritmo de Kruskal, el sistema evalúa los caminos del más barato al más caro. Las líneas continuas y gruesas representan las aristas aceptadas (como la conexión B-C con peso 1) por ser seguras, mientras que las líneas punteadas en rojo (como la conexión A-C con peso 3) representan conexiones que el algoritmo omitió inteligentemente porque habrían formado un ciclo cerrado. El resultado es un árbol perfecto con un costo total minimizado de 13.

4. Problema del Agente Viajero (Traveling Salesperson Problem - TSP)

¿Qué es?

Es la evolución natural (y mucho más compleja) del problema de recorrer un grafo. Imagina a un vendedor que debe visitar una lista de ciudades para vender sus productos y, al final del día, regresar a su casa. El TSP pregunta: ¿Cuál es el orden de ciudades que debe visitar para que la distancia total recorrida sea la más corta posible?

A diferencia del MST (que solo busca conectar todo en forma de ramificaciones), el TSP exige crear una **ruta continua**, un anillo que visite todo exactamente una vez. Esto lo convierte en un problema de permutaciones. Pertenece a la clase NP-difícil, y es famoso por la **"explosión combinatoria"**: para 10 ciudades hay unas 181 mil rutas; para 20 ciudades, hay más de 6×10^{16} rutas, lo que hace inútil el intentar calcular todas las opciones.

Reglas y Limitaciones (Restricciones)

1. **Visita Única (Exclusividad):** Cada ciudad debe ser visitada exactamente una vez a lo largo del recorrido.
2. **Ciclo Hamiltoniano:** La ruta debe iniciar en una ciudad origen, atravesar el resto de los vértices y volver de manera obligatoria a la ciudad de origen. No es una línea recta, es un ciclo cerrado.
3. **Costos Totales (Grafo completo):** Generalmente se asume un grafo completo (todas las ciudades están conectadas con todas). Si la distancia de A a B es igual que de B a A, se le llama *TSP Simétrico*. Si no lo es (por ejemplo, calles de un solo sentido), es un *TSP Asimétrico*. La desigualdad triangular (ir de A a C directamente siempre es menor o igual que ir de A a B y luego a C) suele cumplirse en versiones espaciales (Euclidianas) del problema.

Objetivo Principal

El objetivo es encontrar la permutación de vértices (el recorrido) que **minimice la distancia (o costo) total** del ciclo. Debido a que encontrar la solución óptima absoluta es inviable computacionalmente para instancias grandes, el objetivo práctico de las investigaciones en TSP suele enfocarse en desarrollar **Heurísticas y Metaheurísticas**. El objetivo real es: ¿Cómo encuentro una ruta que esté a menos de un 1% o 2% de diferencia de la ruta matemáticamente perfecta, pero resolviéndolo en segundos en lugar de años?

Representación del problema



La representación del TSP se muestra comparando dos posibles ciclos para visitar un mapa de ubicaciones (nodos A hasta J). En el primer escenario, se traza una ruta aleatoria que resulta en un ciclo desordenado, lleno de cruces ineficientes y una alta distancia total de 2189 pixeles. En contraste, en el segundo escenario se aplica la heurística del "Vecino Más Cercano", la cual genera un polígono o ruta mucho más lógica y limpia. Esta representación gráfica demuestra cómo un buen algoritmo cumple el objetivo de minimizar los costos, reduciendo la distancia del viaje a solo 1204 pixeles (una mejora del 45%).